

Reviewer Pack — BIBD Incidence Matrix ACC⁰ Circuit Size Lower Bound

Entry f7b05c9ca4f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 08:23:28 UTC

BIBD Incidence Matrix ACC⁰ Circuit Size Lower Bound

Entry ID: f7b05c9ca4f6 Verdict: INCONCLUSIVE Recorded: 2026-05-10 08:23:28 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Designs (Balanced Incomplete Block Designs) **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For any Balanced Incomplete Block Design (BIBD) with parameters (v, k, λ) , the corresponding incidence matrix function $f: \{0,1\}^v \rightarrow \{0,1\}$ requires ACC⁰ circuits of size $\Omega(v^{\lfloor 2 - 1/k \rfloor})$.

Rationale (proposer’s reasoning):

The regular structure of BIBDs imposes constraints on the incidence matrix that make it resistant to efficient ACC⁰ computation, leading to a lower bound on circuit size.

Taxonomy category: ACC_SIPSER (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e5137cbdcf447e0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(min_val, max_val):
    primes = []
    for num in range(min_val, max_val + 1):
        if is_prime(num):
            primes.append(num)
    return primes

def generate_bibd(v, k, lambd):
    blocks = set()
    while len(blocks) < v:
        block = set(random.sample(range(v), k))
        if all(block & b == set() for b in blocks):
            blocks.add(frozenset(block))
    blocks = [list(b) for b in blocks]
    incidence_matrix = [[1 if i in block else 0 for block in blocks] for i in range(v)]
    return incidence_matrix, len(blocks)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_tests = 30
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(n_tests):
        v = random.randint(5, 40)
        k = random.randint(2, v // 2)
        lambd = random.randint(1, min(v - k + 1, 3))
        incidence_matrix, num_blocks = generate_bibd(v, k, lambd)

        if num_blocks != (v * (k - 1)) // (lambd * (k - 1)):
            counterexample = "Parameters do not form a valid BIBD"
            conjecture_holds = False
```

```

        break

    # Estimate ACC^0 circuit size using known lower bounds
    expected_circuit_size = v ** (2 - 1 / k)
    actual_circuit_size = num_blocks * k

    if actual_circuit_size < expected_circuit_size:
        counterexample = f"Expected at least {expected_circuit_size}, got {actual_circuit_size}"
        conjecture_holds = False
        break

    total_metric_value += actual_circuit_size
    instances_tested += 1

    return {
        "metric_name": "ACC^0 Circuit Size",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else None,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else generate_primes(2, 30)
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) /
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["counterexample"] == "" for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(r["counterexample"] != "" for r in results):
        first_failing_seed = next((r["seed"] for r in results if r["counterexample"] != ""), None)
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109250
2	propose	ollama_remote	qwen3:8b	0	0	92141
3	preregistration	ollama_remote	qwen3:8b	0	0	24043
4	novelty	ollama_remote	qwen3:8b	0	0	21398
5	novelty	ollama_remote	qwen3:8b	0	0	13499
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12912
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
8	judge	ollama_remote	qwen3:8b	0	0	17658

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 300795 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/f7b05c9ca4f6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f7b05c9ca4f6.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f7b05c9ca4f6.tar.gz (if generated)